

Pronalaženje praznog pravougaonika maksimalne površine

Problem koji se rešava

U ovom projektu je za dati skup tačaka u ravni sa pozitivnim celobrojnim koordinatama potrebno odrediti pravougaonik najveće površine čije su stranice paralelne koordinatnim osama, a temena pripadaju zadatom skupu. Pravougaonik se smatra validnim ako se na njegovim ivicama ili u unutrašnjosti ne nalazi nijedna tačka osim samih temena.

Naivno rešenje

Najjednostavniji način rešavanja ovog problema je provera svih potencijalnih pravougaonika direktnom pretragom. Pravougaonik čije su stranice paralelne koordinatnim osama je jednoznačno određen parom naspramnih temena (x_1, y_1) i (x_2, y_2) , pri čemu postoji $O(N^2)$ takvih parova.

Za svaki par se najpre proverava da su preostala dva temena u skupu tačaka. Ukoliko sva četiri temena postoje, potrebno je proveriti da li je pravougaonik prazan. Provera se vrši linearnim prolaskom kroz preostale tačke i direktnim ispitivanjem da li se neka od njih nalazi u pravougaoniku ili na njegovim ivicama.

Pošto je provera svakog kandidata složenosti $O(N)$, ukupna vremenska složenost ovog algoritma u najgorem slučaju iznosi $O(N^3)$.

Predloženo rešenje

Umesto linearne pretrage algoritam kombinuje tehniku brišuće prave (sweep-line) i perzistentno segmentno stablo, čime se provera kandidata izvršava u logaritamskom vremenu.

Algoritam se sastoji iz sledećih koraka:

- Kako koordinate tačaka mogu imati proizvoljno velike vrednosti, vrši se kompresija X koordinata. Sve jedinstvene X koordinate se sortiraju i preslikaju u kompaktan opseg $[0, m-1]$ (gde je m broj različitih X koordinata u skupu tačaka). Ovim se dubina segmentnog stabla strogo ograničava na $\lceil \log_2 m \rceil$ i minimizuje memorijska složenost.
- Tačke se grupišu i sortiraju rastuće prema Y koordinatama. Brišuća prava se kreće odozdo nagore, zaustavljajući se samo na Y koordinatama gde se nalaze tačke. Svaka takva visina se posmatra kao diskretni trenutak u vremenu gde se prvo proveravaju potencijalni pravougaonici kojima je to gornja ivica, a zatim se tačke ubacuju u stablo i kreira se nova verzija segmentnog stabla.
- Perzistentno segmentno stablo nad kompresovanim X koordinatama održava broj tačaka na odgovarajućim pozicijama. Umesto kreiranja novog stabla iz početka za svaku visinu Y, koristi se perzistentnost zasnovana na deljenju čvorova. Pri dodavanju tačke na nekoj visini, alociraju se novi čvorovi isključivo duž putanje od korena do odgovarajućeg lista (njih $O(\log m)$). Sva ostala podstabla koja ostaju neizmenjena dele se sa prethodnom verzijom pomoću pokazivača. Koren novonastale verzije stabla se pamti u mapi $rootAtY[y]$ i time se istorija stanja čuva uz optimalan utrošak memorije.

Ispitivanje validnosti pravougaonika

Posmatraju se kandidati formirani od uzastopnih parova tacaka (x_1, x_2) na trenutnoj visini y , koji su se kao par vec pojavili na nekoj ranijoj visini y_{bottom} . Validnost pravougaonika $[x_1, x_2] \times [y_{bottom}, y]$ se proverava u dva koraka:

1. Proveravaju se vertikalne ivice u složenosti $O(1)$. Pomoću strukture `lastY_at_x` proverava se da li postoji međutacka na vertikalnim ivicama pravougaonika (poslednja viđena tacka za obe koordinate mora biti y_{bottom}).
2. Ako ivice ne sadrže međutacke proverava se unutrašnjost pravougaonika vremenske složenosti $O(\log m)$. Posmatra se opseg kolona strogo između x_1 i x_2 . Ako između njih uopšte nema drugih X koordinata u ravni, unutrašnjost je trivijalno prazna. U suprotnom, broj tačaka unutar pravougaonika dobija se oduzimanjem dva upita nad stablom: od broja tačaka u opsegu $[x_1, x_2]$ zabeleženih pre tekuće visine y , oduzima se broj tačaka zabeleženih na nivou donje ivice y_{bottom} u istom opsegu. Pravougaonik je validan ako i samo ako je ova razlika jednaka nuli.

Složenost

Vremenska složenost

Sortiranje tačaka i kompresija koordinata traju $O(N \log N)$. Izgradnja perzistentnog stabla zahteva $O(N \log m)$, jer za svaku od N tačaka kreiramo najviše $O(\log m)$ novih čvorova. Provera vertikalnih ivica se vrši u konstantnoj vremenskoj složenosti, dok upiti nad stablom traju $O(\log m)$. Zbog toga je ukupna vremenska složenost algoritma $O(N \log N)$, što je znatno brže od naivnog $O(N^3)$ pristupa.

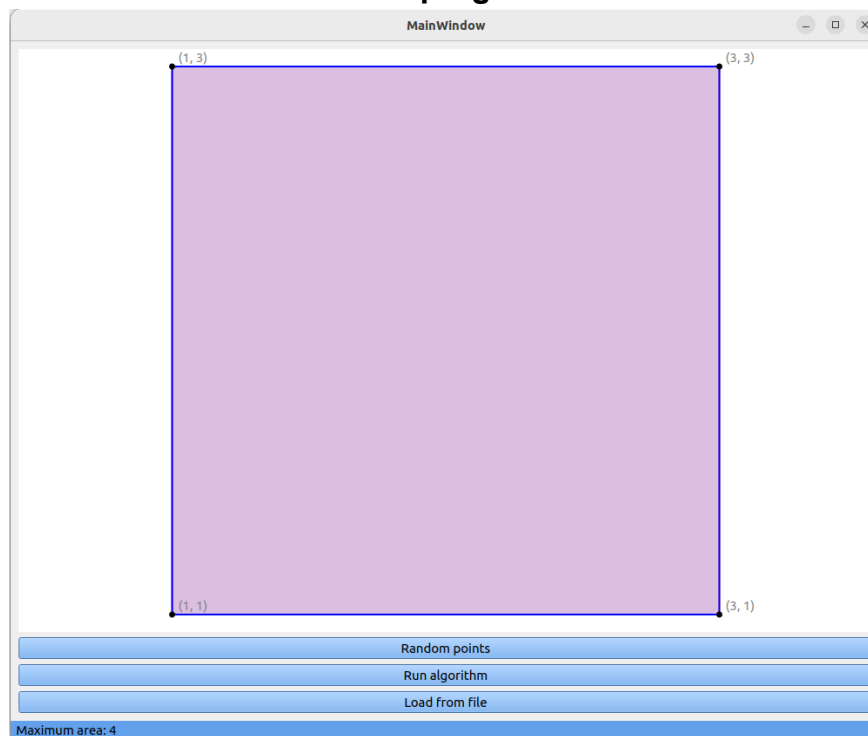
Prostorna složenost

Za svaku tačku u stablo dodajemo najviše $O(\log m)$ novih čvorova, dok pomoćne strukture (`events`, `compX`, `lastY_at_x`, `rootAtY`, `seg`) zauzimaju linearan prostor. Ukupna memorijska složenost je $O(N \log m)$.

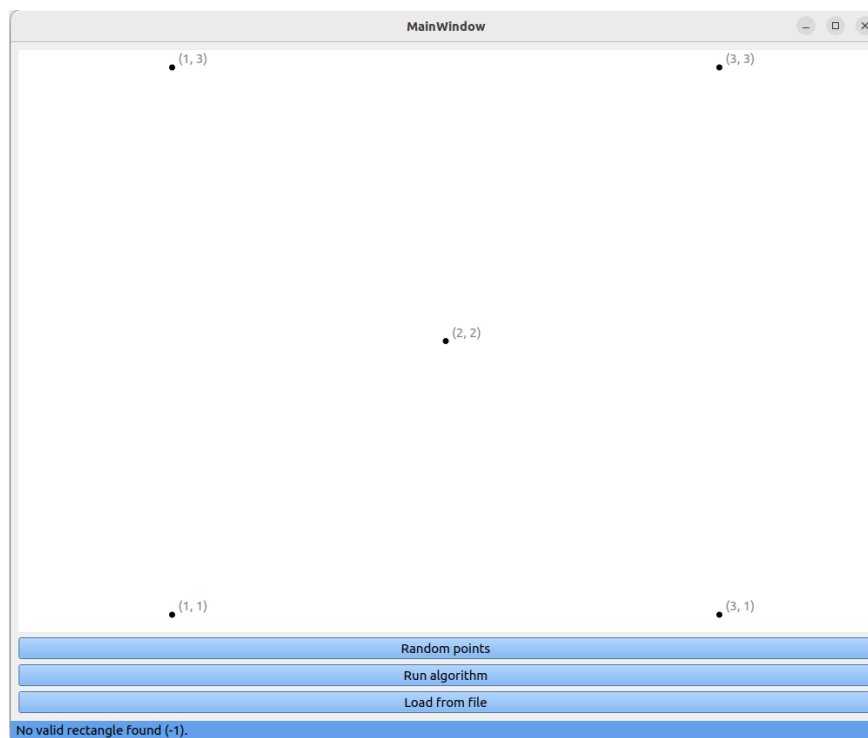
Implementacija

Projekat je implementiran u programskom jeziku C++ uz korišćenje biblioteke Qt za grafički korisnički interfejs i vizuelni prikaz rada algoritma. Korisnički interfejs omogućava nasumično generisanje tačaka, učitavanje koordinata iz tekstualne datoteke i vizuelno praćenje kretanja brišuće prave i isticanje pronađenih pravougaonika. U nastavku su prikazani neki primeri rada programa u različitim scenarijima.

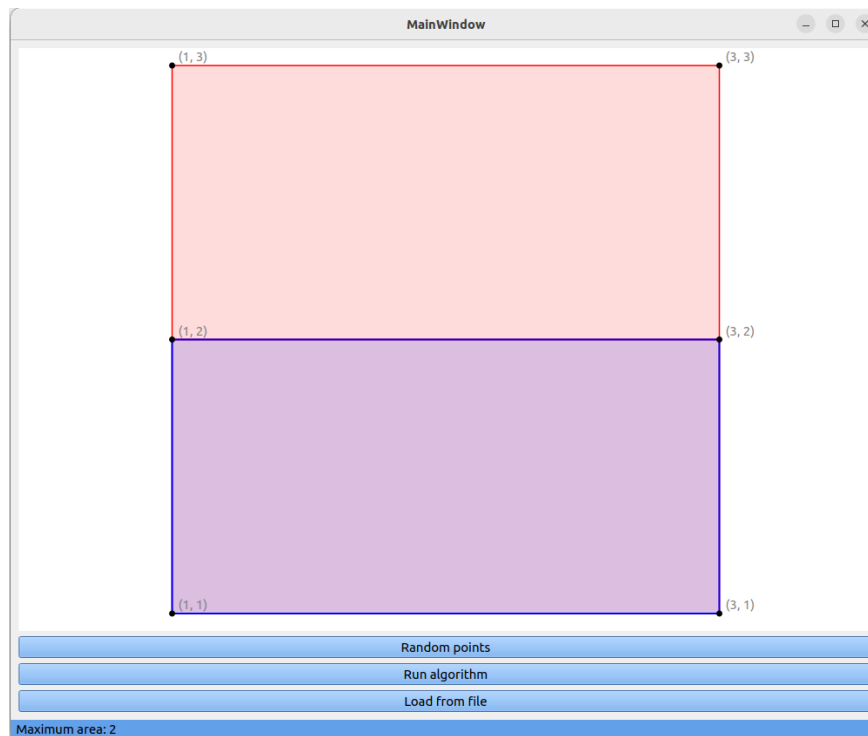
Primeri rada programa



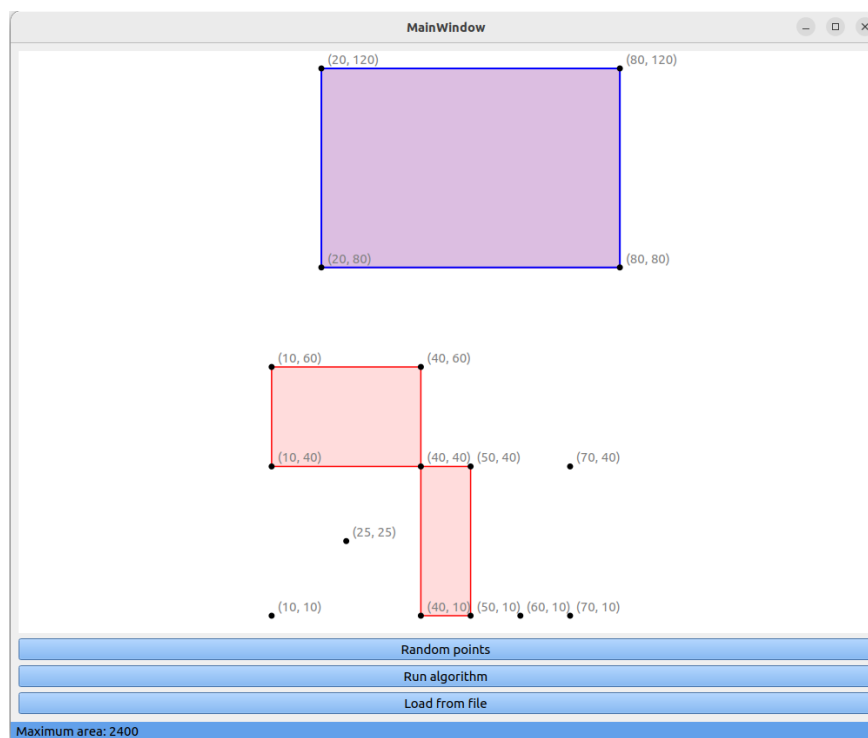
Primer 1: Algoritam pronalazi pravougaonik površine 4



Primer 2: Nema validnih pravougaonika i algoritam ispisuje adekvatnu poruku (-1)



Primer 3: Pronalaženje dva pravougaonika jednake površine (2), s obzirom da su oba pravougaonika jednaka, obojio je prvi na koji je našao odgovarajućom bojom



Primer 4: Algoritam pronalazi sva tri validna pravougaonika, najveći boji odgovarajućom bojom i ispisuje površinu (2400)

Literatura:

<https://poincare.matf.bg.ac.rs/~predrag.janivic/courses/ga.pdf>

<https://faculty.matf.bg.ac.rs/p/jelena.markovic/kurs-geometrijski-algoritmi>

https://tezefiles.matf.bg.ac.rs/master_files/master_radovi/KRI/1079.2022/masterMirjanaJocovic.pdf

<https://www.geeksforgeeks.org/dsa/segment-tree-data-structure/>